

## Week - 31 module

### Q-1 merge sort(easy)

Code:

```
import java.util.*;  
  
public class MergeSortEasy {  
    public static void main (String [] args) {  
        Scanner sc = new Scanner (System.in);  
        int n = sc.nextInt();  
        int [] arr = new int [n];  
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();  
        sort (arr, 0, n-1);  
        for (int n : arr) System.out.print (n + " ");  
    }  
    static void sort (int[] arr, int l, int r) {  
        if (l >= r) return;
```



```
int m = 1 + (n - 1) / 2;
```

```
sort(arr, l, m);
```

```
sort(arr, m + 1, n);
```

```
merge(arr, l, m, n);
```

```
}
```

```
static void merge (int[] arr,  
int l, int m, int r) {
```

```
int[] x = new int[r - l + 1];
```

```
int i = l, j = m + 1, k = 0;
```

```
while (i <= m && j <= r) {  
    (arr[i] <= arr[j]) ? arr[k++] = arr[i++] :
```

```
arr[k++] = arr[j++];
```

```
while (i <= m) arr[k++] = arr[i++];
```

```
while (j <= r) arr[k++] = arr[j++];
```

```
for (i = 0; i < t.length; i++)
```

```
arr[l + i] = x[i];
```

```
}
```

```
}
```

```
MergeSortEasy.java X
1 package week3module;
2
3 import java.util.*;
4
5 public class MergeSortEasy {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int n = sc.nextInt();
10        int[] arr = new int[n];
11        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
12
13        sort(arr, 0, n - 1);
14        for (int x : arr) System.out.print(x + " ");
15    }
16
17    static void sort(int[] arr, int l, int r) {
18        if (l >= r) return;
19        int m = l + (r - l) / 2;
20        sort(arr, l, m);
21        sort(arr, m + 1, r);
22        merge(arr, l, m, r);
23    }
24
25    static void merge(int[] arr, int l, int m, int r) {
26        int[] t = new int[r - l + 1];
27        int i = l, j = m + 1, k = 0;
28        while (i <= m && j <= r) t[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
29        while (i <= m) t[k++] = arr[i++];
30        while (j <= r) t[k++] = arr[j++];
31        for (i = 0; i < t.length; i++) arr[l + i] = t[i];
32    }
33 }
```

Javadoc Declaration Console X

<terminated> MergeSortEasy [Java Application] C:\Users\chaud.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86\_64\_21.0.11.v20260515-1531\jre\bin\javaw.exe (20-Jul-2026, 12:06)

6 450 1200 800 650 300 1500  
300 450 650 800 1200 1500



## 2) Merge Sort Medium

```
import java.util.*;  
  
public class MergeSortMedium {  
    public static void main (String[] args)  
    {  
        Scanner sc = new Scanner(System.in);  
  
        int n = sc.nextInt();  
  
        int[] arr = new int[n];  
  
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();  
  
        sort(arr, 0, n-1);  
  
        for (int x: arr) System.out.print  
            (x + " ");  
        System.out.println();  
  
        int median = arr[n/2];  
  
        int count = 0;  
        for (int n: arr) if (n > median)  
            count++;  
    }  
}
```



```
System.out.println("Median: " + median);
```

```
System.out.println("Order Above  
median: " + count);
```

```
System.out.println("Difference"  
(arr[n-1] - arr[0]));
```

```
Static void sort (int[] arr,  
int l, int r) {
```

```
if (l >= r) return;
```

```
int m = l + (r - l) / 2;
```

```
sort (arr, l, m);
```

```
sort (arr, m + 1, r);
```

```
merge (arr, l, m, r);
```

```
}
```

```
Static void merge (int[] arr,
```

```
int l, int m, int r) {
```

```
int t = new int[n-1+1];
```

```
int i = 1, j = m+1, k = 0;
```

```
while (i <= m && j <= n) t[k++] = (arr[i]  
    <= arr[j] ? arr[i++] : arr[j++]);
```

```
while (i <= m) t[k++] = arr[i++];
```

```
while (j <= n) t[k++] = arr[j++];
```

```
for (i = 0; i < t.length; i++) arr[i+i] =  
    t[i];
```

```
}
```

```
}
```



```
1 package week3module;
2 import java.util.*;
3
4 public class MergeSortMedium {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         int n = sc.nextInt();
8         int[] arr = new int[n];
9         for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
10
11         sort(arr, 0, n - 1);
12         for (int x : arr) System.out.print(x + " ");
13         System.out.println();
14
15         int median = arr[n / 2];
16         int count = 0;
17         for (int x : arr) if (x > median) count++;
18
19         System.out.println("Median: " + median);
20         System.out.println("Orders Above Median: " + count);
21         System.out.println("Difference: " + (arr[n - 1] - arr[0]));
22     }
23
24     static void sort(int[] arr, int l, int r) {
25         if (l >= r) return;
26         int m = l + (r - l) / 2;
27         sort(arr, l, m);
28         sort(arr, m + 1, r);
29         merge(arr, l, m, r);
30     }
31 }
```

```

31
32 static void merge(int[] arr, int l, int m, int r) {
33     int[] t = new int[r - l + 1];
34     int i = l, j = m + 1, k = 0;
35     while (i <= m && j <= r) t[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
36     while (i <= m) t[k++] = arr[i++];
37     while (j <= r) t[k++] = arr[j++];
38     for (i = 0; i < t.length; i++) arr[l + i] = t[i];
39 }
40 }

```

[Javadoc](#)
[Declaration](#)
[Console](#)

<terminated> MergeSortMedium [Java Application] C:\Users\chaud\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86\_64\_21.0.11.v2026051

7 12 25 18 30 15 22 40

12 15 18 22 25 30 40

Median: 22

Orders Above Median: 3

Difference: 28



3> Quick sort (easy)

import java.util.\*;

public class QuickSortEasy {

public static void main (String[] args) {

Scanner sc = new Scanner (System.in);

int n = sc.nextInt();

int [] arr = new int [n];

for (int i = 0; i < n; i++)

arr[i] = sc.nextInt();

quickSort (arr, 0, n-1);

for (int u : arr) System.out.

print (u + " ");

}

static void quickSort (int [] arr,  
int low, int high) {



```
if (low >= high) return;
```

```
int pi = partition(arr, low, high);
```

```
quick sort (arr, low, pi - 1);
```

```
quick sort (arr, pi + 1, high);  
}
```

```
Static int partition (int [] arr, int low  
int high) {
```

```
int pivot = arr [high] ; i = low - 1;
```

```
for (int j = low; j < high; j++) {
```

```
if (arr [j] < pivot) {  
i++
```

```
int temp = arr [i];
```

```
arr [i] = arr [j]; arr [j] = temp;
```

```
}  
}
```

```
int temp = arr [i + 1];
```

```
arr [i + 1] = arr [high];
```

```
arr [high] = temp;
```

```
return i + 1;
```

```
}
```

```
}
```



```

1 package week3module;
2 import java.util.*;
3 public class QuickSortEasy {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         int n = sc.nextInt();
7         int[] arr = new int[n];
8         for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
9
10        quickSort(arr, 0, n - 1);
11        for (int x : arr) System.out.print(x + " ");
12    }
13    static void quickSort(int[] arr, int low, int high) {
14        if (low >= high) return;
15        int pi = partition(arr, low, high);
16        quickSort(arr, low, pi - 1);
17        quickSort(arr, pi + 1, high);
18    }
19    static int partition(int[] arr, int low, int high) {
20        int pivot = arr[high], i = low - 1;
21        for (int j = low; j < high; j++) {
22            if (arr[j] < pivot) {
23                i++;
24                int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
25            }
26        }
27        int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
28        return i + 1;
29    }
30 }

```

 Javadoc
  Declaration
  Console
 

<terminated> QuickSortEasy [Java Application] C:\Users\chaud\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86\_64\_21.0.11.v20260515-1531\jre\bin\javaw.exe

6 120 45 78 200 60 95

45 60 78 95 120 200

#### 4 quicksort medium

```
import java.util.*;

public class quicksortMedium {

    public static void main (String[] args) {

        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt();
        long [] arr = new new long [n];
        long total sum = 0;
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextLong();

            total sum += arr[i];
        }
        quicksortDesc (arr, 0, n-1);

        for (long x: arr) System.out.print(x + " ");
        System.out.println();
        System.out.print ("Top 5:");
    }
}
```



```
long top5sum = 0;
```

```
for (int i = 0; i < 5; i++) {
```

```
    system.out.print (" " + arr[i]);  
    top5sum += arr[i];
```

```
}
```

```
system.out.println();
```

```
system.out.println ("Average of Top 5 : "  
                    + (top5sum/5));
```

```
double overallAvg = (double) totalsum/n;
```

```
int count = 0;
```

```
for (long n : arr) if (n > overallAvg) count++
```

```
system.out.println ("Value Above overall  
Average : " + count);
```

```
}
```

```
static void quickSortDesc (long [] arr,  
    int low, int high) {  
    if (low >= high) return;
```

```
    int pi = partitionDesc (arr, low, high);
```

```
    quickSortDesc (arr, low, pi-1);
```



Date \_\_\_\_\_  
Page \_\_\_\_\_

```
quicksort_desc(arr, p+1, high);  
}
```

```
static int partition_desc(long arr,  
int low, int high) {
```

```
    long pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)  
    {
```

```
        if (arr[j] > pivot)  
            i++;
```

```
        long temp = arr[i];
```

```
        arr[i] = arr[j];
```

```
        arr[j] = temp;
```

```
    }
```

```
}
```

```
    long temp = arr[i+1];
```

```
    arr[i+1] = arr[high];
```

```
    arr[high] = temp;
```

```
    return i+1;
```

```
}
```

```
}
```



```
1 package week3module;
2 import java.util.*;
3 public class QuickSortMedium {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         int n = sc.nextInt();
7         long[] arr = new long[n];
8         long totalSum = 0;
9         for (int i = 0; i < n; i++) {
10             arr[i] = sc.nextLong();
11             totalSum += arr[i];
12         }
13
14         quickSortDesc(arr, 0, n - 1);
15         for (long x : arr) System.out.print(x + " ");
16         System.out.println();
17
18         System.out.print("Top 5:");
19         long top5Sum = 0;
20         for (int i = 0; i < 5; i++) {
21             System.out.print(" " + arr[i]);
22             top5Sum += arr[i];
23         }
24         System.out.println();
25
26         System.out.println("Average of Top 5: " + (top5Sum / 5));
27
28         double overallAvg = (double) totalSum / n;
29         int count = 0;
30         for (long x : arr) if (x > overallAvg) count++;
31         System.out.println("Values Above Overall Average: " + count);
32     }
```

```

32 }
33
34 static void quickSortDesc(long[] arr, int low, int high) {
35     if (low >= high) return;
36     int pi = partitionDesc(arr, low, high);
37     quickSortDesc(arr, low, pi - 1);
38     quickSortDesc(arr, pi + 1, high);
39 }
40
41 static int partitionDesc(long[] arr, int low, int high) {
42     long pivot = arr[high];
43     int i = low - 1;
44     for (int j = low; j < high; j++) {
45         if (arr[j] > pivot) { // '>' makes it sort descending
46             i++;
47             long temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
48         }
49     }
50     long temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
51     return i + 1;
52 }
53 }

```

[Javadoc](#)
[Declaration](#)
[Console](#)

<terminated> QuickSortMedium [Java Application] C:\Users\chaud\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86\_64\_2

8 500 1200 700 2000 900 1500 3000 2500

3000 2500 2000 1500 1200 900 700 500

Top 5: 3000 2500 2000 1500 1200

Average of Top 5: 2040

Values Above Overall Average: 3



5-> Heap sort is easy

```
import java.util.*;
```

```
public class HeapsortEasy {
```

```
    public static void main (String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        int[] arr = new int[n];
```

```
        for (int i = 0; i < n; i++)
```

```
            arr[i] = sc.nextInt();
```

```
        heapsort(arr);
```

```
        for (int n : arr)
```

```
            System.out.print(n + " ");
```

```
        }
```

```
    static void heapsort (int[] arr) {
```

```
        int n = arr.length;
```

Page \_\_\_\_\_

```
for (int i = n/2 - 1; i >= 0; i--)
```

```
    heapify (arr, n, i);
```

```
for (int i = n - 1; i > 0; i--) {
```

```
    int temp = arr[0];
```

```
    arr[0] = arr[i];
```

```
    arr[i] = temp;
```

```
    heapify (arr, i, 0);
```

```
}
```

```
static void heapify (int[] arr, int n,  
                    int i) {
```

```
    int length = i, l = 2 * i + 1, r = 2 * i + 2;
```

```
    if (l < n && arr[l] > arr[largest])  
        largest = l;
```

```
    if (r < n && arr[r] > arr[largest])
```

```
        largest = r;
```

```
    if (largest != i) {
```



int swap = arr[i];

arr[i] = arr[largest];

arr[largest] = swap;

heapify(arr, n, largest);

4

3

3

```

1 package week3module;
2
3 import java.util.*;
4 public class HeapSortEasy {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         int n = sc.nextInt();
9         int[] arr = new int[n];
10        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
11
12        heapSort(arr);
13        for (int x : arr) System.out.print(x + " ");
14    }
15
16    static void heapSort(int[] arr) {
17        int n = arr.length;
18        for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
19        for (int i = n - 1; i > 0; i--) {
20            int temp = arr[0]; arr[0] = arr[i]; arr[i] = temp;
21            heapify(arr, i, 0);
22        }
23    }
24
25    static void heapify(int[] arr, int n, int i) {
26        int largest = i, l = 2 * i + 1, r = 2 * i + 2;
27        if (l < n && arr[l] > arr[largest]) largest = l;
28        if (r < n && arr[r] > arr[largest]) largest = r;
29        if (largest != i) {
30            int swap = arr[i]; arr[i] = arr[largest]; arr[largest] = swap;
31            heapify(arr, n, largest);
32        }
33    }
34 }

```

Javadoc Declaration Console X

<terminated> HeapSortEasy [Java Application] C:\Users\chaud\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86\_64\_21.0.11.v20260515-1531\jre\bin\ja

6 850 1200 950 600 1500 1000

600 850 950 1000 1200 1500



6) Heapsort Medium

```
import java.util.*;
```

```
public class Heapsortmedium {
```

```
    public static void main (String []  
                             args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        int []arr = new new int[n];
```

```
        double sum = 0;
```

```
        for (int i = 0; i < n; i++) {
```

```
            arr[i] = sc.nextInt();
```

```
            sum += arr[i];  
        }
```

```
        heapsort(arr);
```

```
        for (int n : arr)
```

```
            System.out.print(n + " ");
```

```

system.out.println();

double avg = sum/n;

int count = 0;

for (int n : arr) if (n < avg) count++;

system.out.println("Fastest : " + arr[0]);

system.out.println("Slowest : " + arr[n-1]);

system.out.printlnprintf ("Average : %.2f\n", avg);

system.out.println ("Cases Faster Than  
Average : " + count);

system.out.printf ("Percentage : %.2f  
%.2f %%\n", ((double)  
count / n) * 100);
}

```

```

static void heapSort (int [] arr) {
    int n = arr.length;

    for (int i = n/2 - 1; i >= 0; i--)
        heapify (arr, n, i);
}

```



```
for (int i = n-1; i > 0; i--) {
```

```
    int temp = arr[i];
```

```
    arr[i] = arr[0];
```

```
    arr[0] = temp;
```

```
    heapify(arr, i, 0);
```

```
}
```

```
}
```

```
static void heapify (int[] arr,  
                    int n, int i)
```

```
{ int largest = i, l = 2*i+1, r = 2*i+2;
```

```
  if (l < n && arr[l] > arr[largest])  
      largest = l;
```

```
if (r < n && arr[r] > arr[largest])
```

```
  if (r < n && arr[r] > arr[largest])  
      largest = r;
```

```
  if (largest != i) {
```

```
      int swap = arr[i];
```

$arr[i] = arr[largest];$

$arr[largest] = swap;$

$heapify(arr, n, largest);$

}

}

}



```
1 package week3module;
2
3 import java.util.*;
4
5 public class HeapSortMedium {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int n = sc.nextInt();
10        int[] arr = new int[n];
11        double sum = 0;
12        for (int i = 0; i < n; i++) {
13            arr[i] = sc.nextInt();
14            sum += arr[i];
15        }
16
17        heapSort(arr);
18        for (int x : arr) System.out.print(x + " ");
19        System.out.println();
20
21        double avg = sum / n;
22        int count = 0;
23        for (int x : arr) if (x < avg) count++;
24
25        System.out.println("Fastest: " + arr[0]);
26        System.out.println("Slowest: " + arr[n - 1]);
27        System.out.printf("Average: %.2f\n", avg);
28        System.out.println("Cases Faster Than Average: " + count);
29        System.out.printf("Percentage: %.2f%%\n", ((double) count / n) * 100);
30    }
31}
```

```

31
32● static void heapSort(int[] arr) {
33     int n = arr.length;
34     for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
35●   for (int i = n - 1; i > 0; i--) {
36       int temp = arr[0]; arr[0] = arr[i]; arr[i] = temp;
37       heapify(arr, i, 0);
38   }
39 }
40
41● static void heapify(int[] arr, int n, int i) {
42     int largest = i, l = 2 * i + 1, r = 2 * i + 2;
43     if (l < n && arr[l] > arr[largest]) largest = l;
44     if (r < n && arr[r] > arr[largest]) largest = r;
45●   if (largest != i) {
46       int swap = arr[i]; arr[i] = arr[largest]; arr[largest] = swap;
47       heapify(arr, n, largest);
48   }
49 }
50 }

```

[Javadoc](#)
[Declaration](#)
[Console](#)

<terminated> HeapSortMedium [Java Application] C:\Users\chaud\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86\_64\_21.0.11.v20260515-1531\jre\bin\javaw.exe (G

8 12 7 15 9 20 5 10 8

5 7 8 9 10 12 15 20

Fastest: 5

Slowest: 20

Average: 10.75

Cases Faster Than Average: 5

Percentage: 62.50%